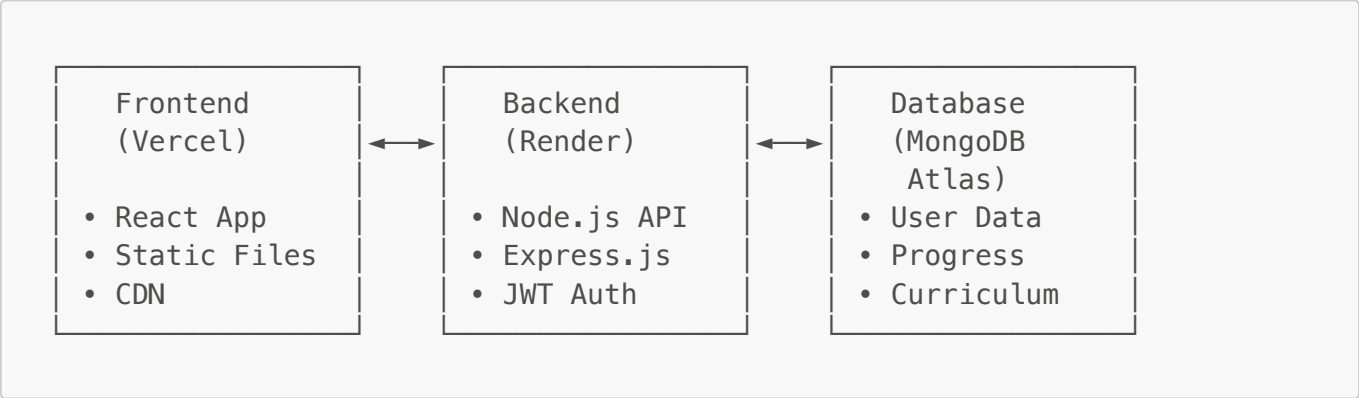
Syncner Deployment Guide

📋 Table of Contents

1. Production Architecture
2. Environment Setup
3. Frontend Deployment (Vercel)
4. Backend Deployment (Render)
5. Database Setup (MongoDB Atlas)
6. Domain Configuration
7. SSL and Security
8. Monitoring and Logging
9. Backup and Recovery
10. Performance Optimization

🏗️ Production Architecture

Overview



Services Used

- **Frontend**: Vercel (Static hosting with CDN)
- **Backend**: Render (Node.js hosting)
- **Database**: MongoDB Atlas (Cloud database)
- **File Storage**: Local storage on Render
- **AI Service**: Google Gemini API
- **Monitoring**: Built-in Vercel/Render analytics

🔧 Environment Setup

Required Environment Variables

Backend (.env)

```
# Application
NODE_ENV=production
```

```
PORT=5000

# Database
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/syncner?
retryWrites=true&w=majority

# Authentication
JWT_SECRET=your-super-secure-jwt-secret-key-min-32-chars
JWT_EXPIRE=30d

# AI Integration
GEMINI_API_KEY=your-gemini-api-key

# CORS
FRONTEND_URL=https://your-frontend-domain.vercel.app

# File Upload
MAX_FILE_SIZE=10485760
UPLOAD_PATH=./uploads

# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100

# Logging
LOG_LEVEL=info
```

Frontend (.env)

```
# API Configuration
VITE_API_URL=https://your-backend-domain.onrender.com
VITE_APP_NAME=Syncner
VITE_APP_VERSION=1.0.0

# Environment
VITE_NODE_ENV=production

# Features
VITE_AI_CHAT_ENABLED=true
VITE_ATTENDANCE_ENABLED=true
VITE_LEADERBOARD_ENABLED=true
```

🚀 Frontend Deployment (Vercel)

1. Prepare Repository

```
# Ensure your repository is clean
git add .
```

```
git commit -m "Production build preparation"
git push origin main
```

2. Connect to Vercel

1. Go to vercel.com
2. Sign in with GitHub
3. Click "New Project"
4. Import your repository
5. Configure project settings

3. Vercel Configuration

Build Settings

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "installCommand": "npm install",
  "framework": "vite"
}
```

vercel.json Configuration

```
{
  "version": 2,
  "builds": [
    {
      "src": "package.json",
      "use": "@vercel/static-build",
      "config": {
        "distDir": "dist"
      }
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "/index.html"
    }
  ],
  "env": {
    "VITE_API_URL": "https://your-backend-domain.onrender.com"
  }
}
```

4. Environment Variables in Vercel

1. Go to Project Settings → Environment Variables
2. Add all frontend environment variables
3. Set for Production environment

5. Custom Domain (Optional)

1. Go to Project Settings → Domains
2. Add your custom domain
3. Configure DNS records as instructed

🔧 Backend Deployment (Render)

1. Prepare Backend

```
# Ensure backend is ready
cd backend
npm install --production
```

2. Connect to Render

1. Go to render.com
2. Sign in with GitHub
3. Click "New +" → "Web Service"
4. Connect your repository

3. Render Configuration

Service Settings

- **Name**: syncner-backend
- **Environment**: Node
- **Region**: Choose closest to your users
- **Branch**: main
- **Root Directory**: backend
- **Build Command**: `npm install`
- **Start Command**: `npm start`

Environment Variables

Add all backend environment variables in Render dashboard:

- Go to Environment tab
- Add each variable from the backend .env list

4. Health Check Configuration

```
// Add to backend/app.js
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    uptime: process.uptime()
  });
});
```

5. Auto-Deploy Setup

- Enable auto-deploy on git push
- Set up deployment notifications
- Configure build timeout (default: 10 minutes)

🗄️ Database Setup (MongoDB Atlas)

1. Create MongoDB Atlas Account

1. Go to mongodb.com/cloud/atlas
2. Create free account
3. Create new cluster

2. Cluster Configuration

- **Provider**: AWS/Google Cloud/Azure
- **Region**: Choose closest to your backend
- **Tier**: M0 (Free) for development, M10+ for production
- **Cluster Name**: syncner-cluster

3. Database Access

1. Go to Database Access
2. Create database user:
    - Username: `syncner-user`
    - Password: Generate secure password
    - Database User Privileges: Read and write to any database

4. Network Access

1. Go to Network Access
2. Add IP Address:
    - For development: Add your IP
    - For production: Add `0.0.0.0/0` (all IPs) or Render's IP ranges

5. Connection String

1. Go to Clusters → Connect
2. Choose "Connect your application"

3. Copy connection string
4. Replace `<password>` with your database user password
5. Replace `<dbname>` with `syncner`

6. Database Initialization

```
# Run migrations after deployment
curl -X POST https://your-backend-domain.onrender.com/api/admin/migrate
```

🌐 Domain Configuration

1. Custom Domain Setup

If using custom domains:

Frontend (Vercel)

1. Add domain in Vercel dashboard
2. Configure DNS:

```
Type: CNAME
Name: www
Value: cname.vercel-dns.com

Type: A
Name: @
Value: 76.76.19.61
```

Backend (Render)

1. Add custom domain in Render dashboard
2. Configure DNS:

```
Type: CNAME
Name: api
Value: your-service.onrender.com
```

2. Update Environment Variables

Update frontend environment variables with new domains:

```
VITE_API_URL=https://api.yourdomain.com
```

🔒 SSL and Security

SSL Certificates

- **Vercel**: Automatic SSL with Let's Encrypt
- **Render**: Automatic SSL for custom domains
- **MongoDB Atlas**: SSL enabled by default

Security Headers

Add to backend middleware:

```
// Security headers
app.use((req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('X-XSS-Protection', '1; mode=block');
  res.setHeader('Strict-Transport-Security', 'max-age=31536000;
includeSubDomains');
  next();
});
```

CORS Configuration

```
// CORS setup
const corsOptions = {
  origin: [
    'https://your-frontend-domain.vercel.app',
    'https://your-custom-domain.com'
  ],
  credentials: true,
  optionsSuccessStatus: 200
};
app.use(cors(corsOptions));
```

📊 Monitoring and Logging

1. Application Monitoring

Vercel Analytics

- Built-in analytics for frontend
- Performance monitoring
- Error tracking

Render Monitoring

- Built-in metrics dashboard
- CPU, memory, and response time monitoring
- Log aggregation

2. Custom Logging

```
// Add to backend
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'error.log', level: 'error'
}),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});
```

3. Health Checks

```
// Health check endpoint
app.get('/health', async (req, res) => {
  try {
    // Check database connection
    await mongoose.connection.db.admin().ping();

    res.status(200).json({
      status: 'healthy',
      timestamp: new Date().toISOString(),
      uptime: process.uptime(),
      database: 'connected',
      version: process.env.npm_package_version
    });
  } catch (error) {
    res.status(503).json({
      status: 'unhealthy',
      error: error.message
    });
  }
});
```

💾 Backup and Recovery

1. Database Backups

MongoDB Atlas Automated Backups

1. Enable automated backups in Atlas
2. Set retention period (7 days for free tier)
3. Configure backup frequency

Manual Backup Script

```
#!/bin/bash
# backup.sh
DATE=$(date +%Y%m%d_%H%M%S)
mongodump --uri="$MONGODB_URI" --out="./backups/backup_$DATE"
```

2. File Backup

```
# Backup uploaded files
tar -czf uploads_backup_$(date +%Y%m%d).tar.gz ./uploads/
```

3. Recovery Procedures

```
# Restore database
mongorestore --uri="$MONGODB_URI" ./backups/backup_20240115_120000/

# Restore files
tar -xzf uploads_backup_20240115.tar.gz
```

⚡ Performance Optimization

1. Frontend Optimization

Build Optimization

```
// vite.config.js
export default {
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
          redux: ['@reduxjs/toolkit', 'react-redux'],
          ui: ['framer-motion', '@radix-ui/react-icons']
        }
```

```
      }
    }
  }
};
```

Caching Strategy

```
// Service worker for caching
const CACHE_NAME = 'syncner-v1';
const urlsToCache = [
  '/',
  '/static/js/bundle.js',
  '/static/css/main.css'
];
```

2. Backend Optimization

Database Indexing

```
// Add indexes for better performance
db.users.createIndex({ email: 1 });
db.userprogress.createIndex({ userId: 1, levelId: 1 });
db.attendance.createIndex({ userId: 1, date: 1 });
```

Response Caching

```
// Add caching middleware
const cache = require('memory-cache');

app.get('/api/curriculum', (req, res, next) => {
  const key = `curriculum_${req.query.careerTrack}`;
  const cached = cache.get(key);

  if (cached) {
    return res.json(cached);
  }

  // ... fetch data
  cache.put(key, data, 300000); // 5 minutes
  res.json(data);
});
```

3. CDN Configuration

- Vercel automatically provides CDN
- Configure cache headers for static assets
- Use image optimization for uploaded files

🔄 CI/CD Pipeline

1. GitHub Actions (Optional)

```yaml
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v20
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.ORG_ID }}
          vercel-project-id: ${{ secrets.PROJECT_ID }}
          working-directory: ./
```

2. Automated Testing

```yaml
# .github/workflows/test.yml
name: Run Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Setup Node.js
        uses: actions/setup-node@v2
        with:
          node-version: '18'

      - name: Install dependencies
        run: |
          cd src && npm install
```

```
          cd ../backend && npm install

      - name: Run frontend tests
        run: cd src && npm test

      - name: Run backend tests
        run: cd backend && npm test
```

🚨 Troubleshooting

Common Issues

1. Build Failures

```
# Clear cache and reinstall
rm -rf node_modules package-lock.json
npm install
npm run build
```

2. Database Connection Issues

- Check MongoDB Atlas network access
- Verify connection string
- Check firewall settings

3. Environment Variable Issues

- Verify all required variables are set
- Check variable names (case-sensitive)
- Restart services after changes

4. CORS Issues

- Update CORS origins in backend
- Check frontend API URL configuration
- Verify domain configurations

Monitoring Commands

```
# Check backend health
curl https://your-backend-domain.onrender.com/health

# Check frontend
curl https://your-frontend-domain.vercel.app

# Check database connection
mongosh "mongodb+srv://username:password@cluster.mongodb.net/syncner"
```

📋 Deployment Checklist

Pre-Deployment

- [ ] All environment variables configured
- [ ] Database connection tested
- [ ] Build process verified locally
- [ ] Security headers configured
- [ ] CORS settings updated
- [ ] SSL certificates ready

Post-Deployment

- [ ] Health checks passing
- [ ] Database migrations run
- [ ] User registration/login working
- [ ] File uploads working
- [ ] AI chat integration working
- [ ] Analytics tracking active
- [ ] Backup procedures tested

Monitoring Setup

- [ ] Error tracking configured
- [ ] Performance monitoring active
- [ ] Log aggregation working
- [ ] Alert notifications set up
- [ ] Backup verification scheduled

🔧 Maintenance

Regular Tasks

- **Weekly**: Check error logs and performance metrics
- **Monthly**: Review and update dependencies
- **Quarterly**: Security audit and backup testing
- **Annually**: Full system review and optimization

Updates and Patches

1. Test updates in staging environment
2. Create database backups before updates
3. Deploy during low-traffic periods
4. Monitor system health post-deployment
5. Rollback plan ready if needed

Deployment Version: v1.0
Submitted On: September 2025 **Supported Platforms**: Vercel, Render, MongoDB Atlas